Decision Brief: Live Visualization of Wheat

Date: 2026-03-11 | **Audience**: Developer learning Wheat | **Phase**: Compiled

Executive Summary

The sprint successfully built a live, self-referential visualization of the Wheat framework — a real-time dashboard that teaches Wheat by being a product of it. The approach works: a zero-dependency Node.js SSE server watches `claims.json`, auto-compiles, and pushes updates to a timeline-first browser dashboard. All 5 original constraints are satisfied, 48 claims across 8 topics have been validated, and the system scales to 1000+ claims with sub-60ms compile times.

Recommendation

Ship the current implementation as the canonical Wheat teaching tool. Specific next steps:

1. **Use the live dashboard** (`node wheat-server.js`, port 3141) as the primary dev-time experience during sprints — it demonstrates Wheat's claim→compile→artifact loop in real time [f003, p005].
2. **Use static HTML artifacts** (`output/dashboard.html`, `/status` command) for sharing and stakeholder consumption — fully self-contained, zero dependencies, works via `file://` [f002, p008, e003].
3. **Fix the two remaining robustness gaps**: the compiler now handles malformed JSON gracefully after evaluation-phase fixes [e002].
4. **Adopt the 17-command suite** organized in 3 tiers (Lifecycle, QA, Operations) for future sprints [r013, r016].

Evidence Summary

Architecture (14 claims, tested)

The system architecture is a zero-dependency Node.js stack: `wheat-server.js` (133 lines) serves a single-page dashboard via HTTP with SSE push updates [p001]. `fs.watch` with 150ms debounce provides reliable file change detection on macOS [p004]. SSE auto-reconnect is handled natively by the browser's `EventSource` API — no retry logic needed

[p002]. The compiler (v0.2.0, 685 lines) runs a 7-pass pipeline and scales linearly: 45ms at 42 claims, 59ms at 1000 claims [e001, r014].

Browser-only approaches were ruled out — the File System Access API is Chrome-only and still requires a server for CLI integration [r003]. The HMR-style hash notification pattern keeps SSE payloads lightweight [r006].

The compiler expanded from 10 to 17 commands in 3 tiers, with 9 new capabilities including `--diff` mode, corroboration tracking, and constraint-aware coverage [r013, r014, r015]. Three QA feedback loops were introduced: self-correction (`/challenge`), external validation (`/witness`), and meta-learning (`/calibrate`) [r016].

UX Pattern (8 claims, tested)

The timeline-first design replaced the original split-pane card grid after stakeholder feedback [f005]. Claims appear as expandable nodes grouped by phase, with progressive disclosure via click-to-expand [p006]. Phase-colored visual system (define=amber, research=blue, prototype=green, evaluate=purple, feedback=cyan) provides instant grouping without labels [p007]. Reverse chronological order puts newest content at top [f006]. The compiler stepper animates across the top on each update [r005, p003].

Accessibility (8 claims, tested)

The dashboard initially had zero accessibility infrastructure [r018, r019, r020]. Five targeted fixes brought it to WCAG 2.2 AA compliance in ~25 lines of changes [r021, p009]:

1. Contrast: `--text-dim` raised from #505a6e (2.78:1) to #7a839a (5.08:1) [r017]
2. Semantic landmarks: `<header>` , `<main>` , `role="status"` , `role="region"` [r018]
3. Keyboard navigation: `tabindex` , `role="button"` , `aria-expanded` , Enter/Space handlers [r019]
4. Live regions: `aria-live="polite"` on scoreboard and timeline for SSE updates [r020]
5. Focus styles: `:focus-visible` outlines on all interactive elements [r021]

All 13 WCAG 2.2 AA checks pass [e004]. The explainer template has a residual external Google Fonts import that should be removed [r022].

Output Format (6 claims, tested)

All 7 HTML artifacts verified fully self-contained: zero external `<script src>`, `<link href>`, `@import url()`, or `<img src>` pointing to external resources [e003, p008]. Sizes range from 7.3KB to 26.3KB. Every file opens correctly via `file://` with no network requests. The live server and static artifacts serve complementary roles: server for dev-time, static for sharing [r007, f002, f003].

Dogfooding (3 claims, tested)

20 of 48 claims are self-referential — they describe the dashboard that displays them [e005]. The sprint exercises 5/5 phases (define, research, prototype, feedback, evaluate) with 16 wheat-format git commits. The self-referential loop is itself the teaching mechanism: a developer reading the dashboard sees claims being made about the dashboard they're reading [p005, d001].

Quality (5 claims, tested)

Evidence distribution: stated 27.5%, web 42.5%, documented 10%, tested 22.5% [e006]. The compiler's constraint-aware coverage correctly identifies that `audience` and `scope` topics are constraint-dominated and don't need evidence upgrades [r009, r011]. The false-positive warning rate was reduced by introducing `W_CONSTRAINT_ONLY` vs `W_WEAK_EVIDENCE` [r012]. Success criteria remains subjective ("I'll know it when I see it") [d005].

Tradeoffs and Risks

Risk	Evidence	Mitigation
Live server conflicts with self-contained HTML constraint	tested [r007]	Two artifacts: server for dev, static for sharing [f002, f003]
Compiler crashes on empty/malformed JSON	tested [e002]	Fixed: try/catch added, clean error messages
Explainer template imports external Google Fonts	web [r022]	Use system font stack; remove `@import`
No scalability testing beyond 1000 claims	tested [e001]	Linear scaling confirmed; 1000 in 59ms is sufficient headroom
Quality topic resistant to evidence upgrade	tested [e006]	Subjective criteria by design; constraint-dominated

Resolved Conflicts

Winner	Loser	Resolution
p001 (tested): Server is 133 lines, zero dependencies	r008 (web): Estimated ~80-100 lines	Evidence tier: tested (4) > web (2). Actual line count exceeded estimate by 33%.

Appendix: Claim Inventory

ID	Type	Topic	Evidence	Content
d001	constraint	dogfooding	stated	Visualization must be built through Wheat pipeline itself
d002	constraint	output-format	stated	Self-contained HTML, no external dependencies
d003	constraint	audience	stated	Single developer, self-explanatory without docs
d004	constraint	scope	stated	Full creative freedom, no budget/timeline constraints
d005	constraint	quality	stated	Subjective success — visually compelling first impression
r002	factual	architecture	web	fs.watch uses OS-level notifications, zero deps needed
r003	factual	architecture	web	Browser-only approaches not viable (Chrome-only)
r004	recommendation	ux-pattern	web	AST Explorer split-pane pattern for input/output
r005	recommendation	ux-pattern	web	GitHub Actions linear pipeline stepper for compiler
r006	recommendation	ux-pattern	web	Next.js HMR hash-notification pattern for SSE
r007	risk	output-format	web	Server vs self-contained tension; two-artifact resolution

r009	factual	quality	web	3/4 weak evidence warnings are false alarms
r010	factual	output-format	web	Output-format is only real evidence gap
r011	risk	quality	web	Compiler warning system has constraint blind spot
r012	recommendation	architecture	web	Add constraint-aware coverage to compiler
r013	factual	architecture	documented	17 commands in 3 tiers
r014	factual	architecture	documented	Compiler v0.2.0: 685 lines, 9 new capabilities
r015	factual	architecture	documented	9 claim ID prefixes
r016	recommendation	architecture	documented	Tier 2 QA creates 3 feedback loops
r017	factual	accessibility	web	--text-dim contrast 2.78:1 fails WCAG AA
r018	risk	accessibility	web	Zero ARIA landmarks or semantic HTML
r019	risk	accessibility	web	No keyboard navigation on claim rows
r020	risk	accessibility	web	No aria-live for SSE updates
r021	recommendation	accessibility	web	Five fixes for WCAG 2.2 AA (~30 lines)
r022	risk	accessibility	web	Template imports external Google Fonts
p001	factual	architecture	tested	Server is 133 lines, zero npm deps
p002	factual	architecture	tested	SSE auto-reconnect confirmed working
p003	factual	ux-pattern	tested	Split-pane layout works

p004	factual	architecture	tested	fs.watch reliable, 150ms debounce, ~50ms compile
p005	factual	dogfooding	tested	Self-referential loop confirmed working
p006	factual	ux-pattern	tested	Timeline-first dashboard with progressive disclosure
p007	factual	ux-pattern	tested	Phase-colored visual system effective
p008	factual	output-format	tested	All 4 HTML artifacts verified self-contained
p009	factual	accessibility	tested	All 5 WCAG 2.2 AA fixes implemented, 13/13 pass
f001	feedback	architecture	stated	User prefers static dashboard over live server
f002	feedback	output-format	stated	Self-contained HTML is the right call
f003	feedback	architecture	stated	Live server valuable as dev-time tool
f004	feedback	quality	stated	Dashboard UI needs UX polish pass
f005	feedback	ux-pattern	stated	Needs timeline-based UI, progressive disclosure
f006	constraint	ux-pattern	stated	Reverse chronological — newest first
e001	factual	architecture	tested	Compiler scales linearly: 59ms at 1000 claims
e002	risk	architecture	tested	2 crash cases on malformed JSON (now fixed)
e003	factual	output-format	tested	7/7 artifacts verified self-contained
e004	factual	accessibility	tested	13/13 WCAG 2.2 AA checks pass

e005	factual	dogfooding	tested	20/48 self-referential claims, 5/5 phases
e006	factual	quality	tested	Evidence distribution: 22.5% tested, all topics covered

---

Compilation certificate: sha256:b311e643ba878... | Compiler: wheat v0.2.0 | Claims: 48 (46 active, 2 superseded) | Compiled: 2026-03-11T08:17:50.289Z